

UR5e Control Analysis

Part 1B — Controller Design, Benchmarking, and Robustness Assessment

PID, Computed Torque, Discrete LQR, and Constrained MPC
Nominal Tracking, Runtime Implementation, Model Mismatch, and Disturbance Rejection

Sooyong Kim

Robotics Engineering Portfolio Project

September 2026

Scope. Part 1B begins from the validated and characterized nonlinear UR5e plant established in the Pre-Part 1 and Part 1A studies. I use one common benchmark to design and compare four controller structures, then deliberately break the nominal assumptions with a persistent payload mismatch and a transient external joint disturbance. The objective is not to declare a universal “best” controller, but to identify what each structure gains, what assumptions it depends on, and what fails first when those assumptions are violated.

Executive Summary

I treated controller comparison as an engineering diagnosis problem rather than a leaderboard. Part 1A had already shown that the UR5e plant is strongly configuration dependent, gravity dominated for the selected moderate-speed benchmark, locally rather than globally represented by a fixed linearization, and comfortably below the adopted torque constraints. Those observations determined the Part 1B sequence: begin with a transparent joint PID baseline, add model cancellation through computed torque, test a fixed-equilibrium optimal feedback law with discrete LQR, and finally introduce finite-horizon prediction and explicit input constraints with MPC.

All four controllers were evaluated on the same 4 s quintic joint-space trajectory, 2 ms sample period, nonlinear rigid-body plant, RK4/ZOH integration, and adopted torque constraints of [150, 150, 150, 28, 28, 28] N m. This kept the comparison focused on controller structure rather than on changing the task. Nominal overall joint RMSE was 0.4698° for PID, 0.001244° for computed torque, 0.01550° for LQR, and 0.04847° for MPC. TCP position RMSE was 5.836, 0.00414, 0.4185, and 0.3784 mm, respectively. These nominal numbers are informative but incomplete: they mainly show how well each controller exploits the assumptions under which it was designed.

The robustness study therefore used two deliberately different perturbations without retuning. R1 added a 2.0 kg point-mass payload whose center of mass was 0.10 m along tool-frame +Z to the *actual plant*, while every controller retained the nominal bare-arm model or gains. R2 applied an unmeasured +20 N m rectangular torque disturbance to Joint 2 from 2.0 to 2.2 s on the nominal plant. The distinction was intentional: R1 is a persistent model mismatch; R2 is a finite-duration external disturbance.

The results separated nominal accuracy from robustness. Computed torque was nearly exact nominally because the controller and plant shared the same model, but its overall joint RMSE increased to 3.684° under the payload mismatch. PID had much larger nominal error, yet its relative degradation under the payload was only 2.57× and its disturbance-case aggregate RMSE was almost unchanged. LQR and MPC both degraded under the persistent payload mismatch; under the specific transient J2 pulse, MPC retained smaller absolute joint and TCP errors than LQR. I do not interpret these observations as universal controller rankings. They are evidence about the tested structures, trajectory, perturbations, and implementation choices.

Implementation was treated as part of the control problem. PID feedback was cross-validated between Python and C++, including a one-sample anti-windup update-order discrepancy found during validation. LQR online feedback was reproduced in C++ to approximately 4.5×10^{-13} N m. For MPC, profiling showed that the original dense Python implementation spent substantial time recomputing terms that were fixed or reference-known. Algebraic offline/online decomposition reduced the mean Python update to about 1.07 ms, and the C++/OSQP implementation reproduced Python torque to 2.03×10^{-10} N m with a mean runtime of about 0.9605 ms. Timing outliers above 2 ms remain, so no hard-real-time claim is made.

1. Scope, Plant Basis, and Common Benchmark

Part 1B starts from the Part 1A plant

$$M(q)\ddot{q} + c(q, \dot{q}) + g(q) = \tau, \quad (1)$$

with state $x = [q^T \ \dot{q}^T]^T$. The model is fixed-base, rigid-body, bare-arm, and excludes friction, compliance, motor electrical dynamics, sensor noise, and the proprietary UR servo. These exclusions are not hidden: they define the analytical layer that Part 2 will later challenge in Isaac Sim.

The common benchmark uses

$$\mathbf{q}_0 = [0, -90, 90, -90, -90, 0]^\circ, \quad \mathbf{q}_f = [20, -60, 60, -70, -70, 20]^\circ,$$

a 4 s quintic joint trajectory, and $T_s = 0.002$ s (500 Hz). Part 1A inverse dynamics showed a nominal peak of approximately 40.79 N m on J2, only 27.2% of the adopted simulation limit. I intentionally retained this moderate trajectory: the nominal comparison should reveal controller behavior rather than collapse into a saturation experiment.

Table 1: Common Part 1B evaluation framework.

Item	Definition
Plant	Nonlinear validated UR5e rigid-body model
Reference	4 s quintic joint-space trajectory
Integration	RK4 with zero-order-held controller command
Sample period	2 ms (500 Hz)
Torque constraints	[150, 150, 150, 28, 28, 28] N m
Tracking metrics	Per-joint RMSE/max/final; overall joint RMSE; TCP position/orientation RMSE/max/final
Effort metrics	Maximum/RMS torque and peak utilization
Implementation metrics	Controller computation time; Python-C++ consistency where implemented
Robustness	R1 persistent payload mismatch; R2 transient J2 torque disturbance; no retuning

2. Engineering Method: Design, Diagnose, Then Escalate Complexity

I did not start with the most sophisticated controller. Each stage was used to expose a specific limitation before adding complexity. The PID establishes what pure error feedback can do without a model. Computed torque asks how far exact nonlinear compensation can improve tracking when the model is correct. LQR asks whether a fixed local model and optimal state feedback can retain good nonlinear trajectory tracking, and the failed first formulation is kept in the report because it revealed a feedforward-structure problem. MPC then adds prediction and constraints, but its implementation is profiled rather than assumed to be computationally acceptable.

This progression also defines how I interpret failures. A poor result is not discarded simply because another tuning could improve it. I first ask whether the error comes from bandwidth, damping, steady disturbance, feedforward structure, model mismatch, numerical formulation, or runtime architecture. Only after identifying the dominant mechanism do I change the controller. This distinction matters because controller tuning can otherwise hide a structural error.

3. Joint PID Baseline

3.1 Formulation and physically interpretable tuning

The joint controller is

$$\boldsymbol{\tau} = K_p \mathbf{e} + K_d \dot{\mathbf{e}} + K_i \int \mathbf{e} dt, \quad \mathbf{e} = \mathbf{q}_{ref} - \mathbf{q}, \quad (2)$$

with conditional integration anti-windup. Rather than choosing six unrelated gain triplets, I used the diagonal of the nominal inertia matrix to construct a physically interpretable initial parameterization from a target natural frequency ω_n , damping ratio ζ , and integral scaling α_i . The final design used $\omega_n = 20$ rad/s, $\zeta = 3$, and $\alpha_i = 18$.

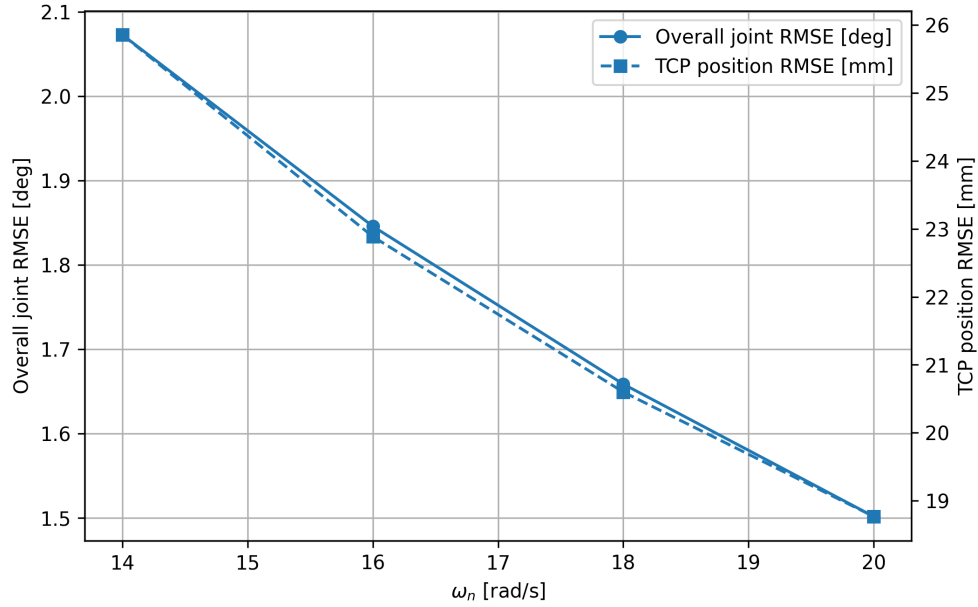


Figure 1: PID bandwidth study. I increased closed-loop bandwidth only after checking the resulting tracking and effort, rather than treating larger gains as automatically better.

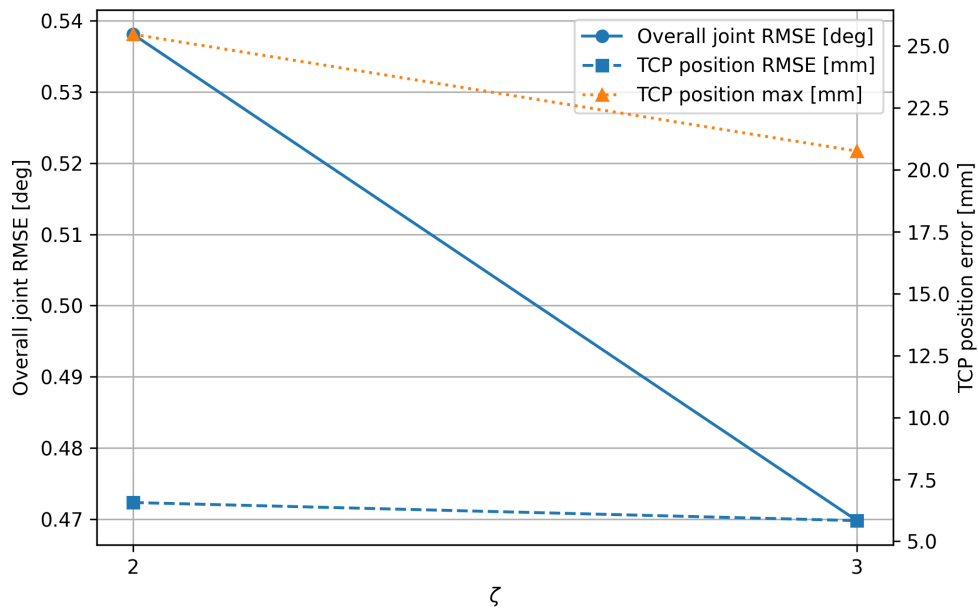


Figure 2: PID damping comparison used to control transient behavior after the bandwidth study.

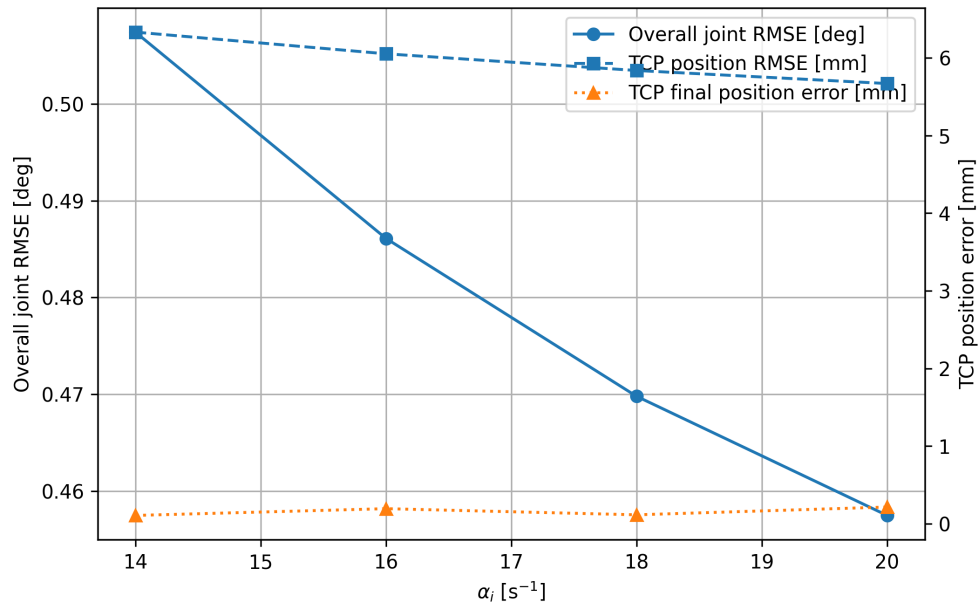


Figure 3: Integral-action study. Integral gain was retained because the uncompensated plant contains a persistent gravity load that proportional–derivative action alone must balance through position error.

3.2 Nominal result and J4 diagnosis

The final PID achieved overall joint RMSE of 0.4698° , TCP position RMSE of 5.836 mm, and TCP orientation RMSE of 1.528° . The peak torque was 40.75 N m on J2 and no joint approached the adopted limit.

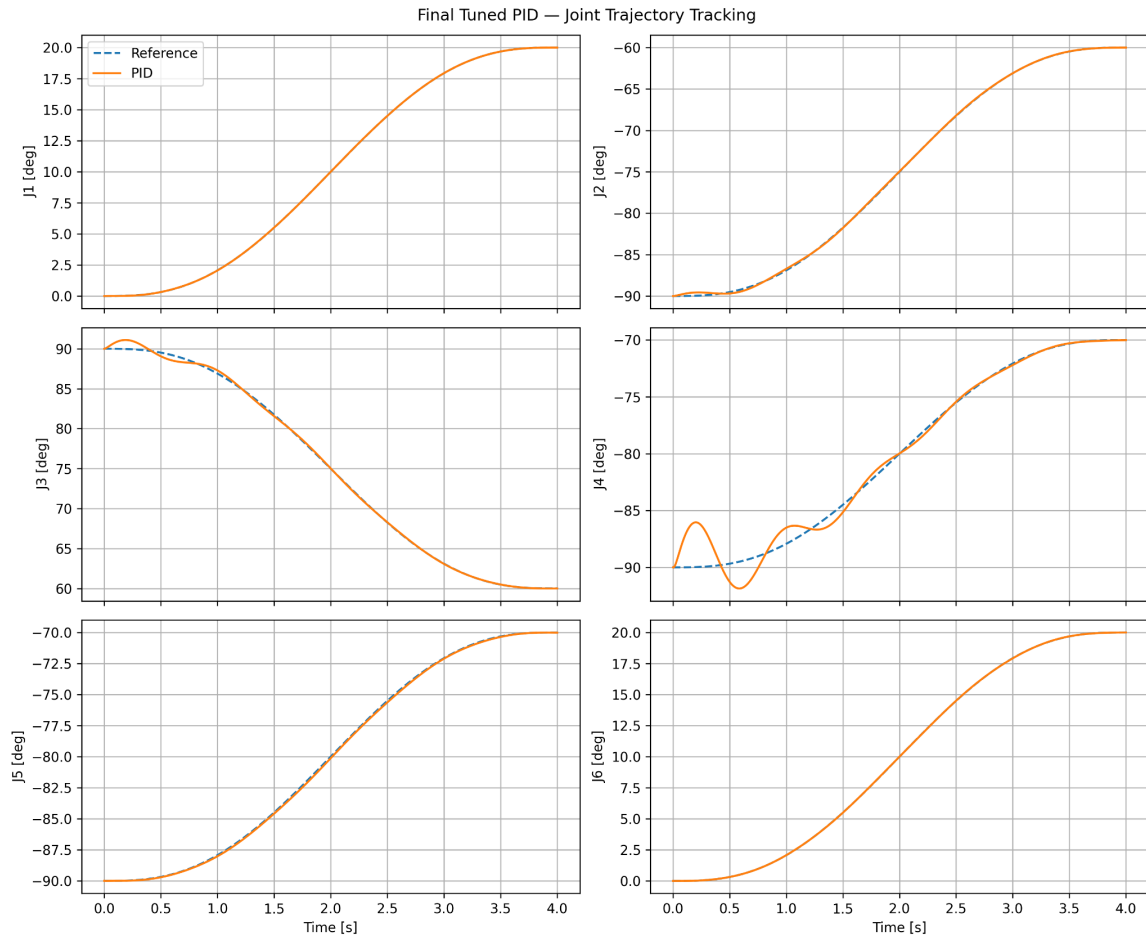


Figure 4: Final nominal PID joint tracking.

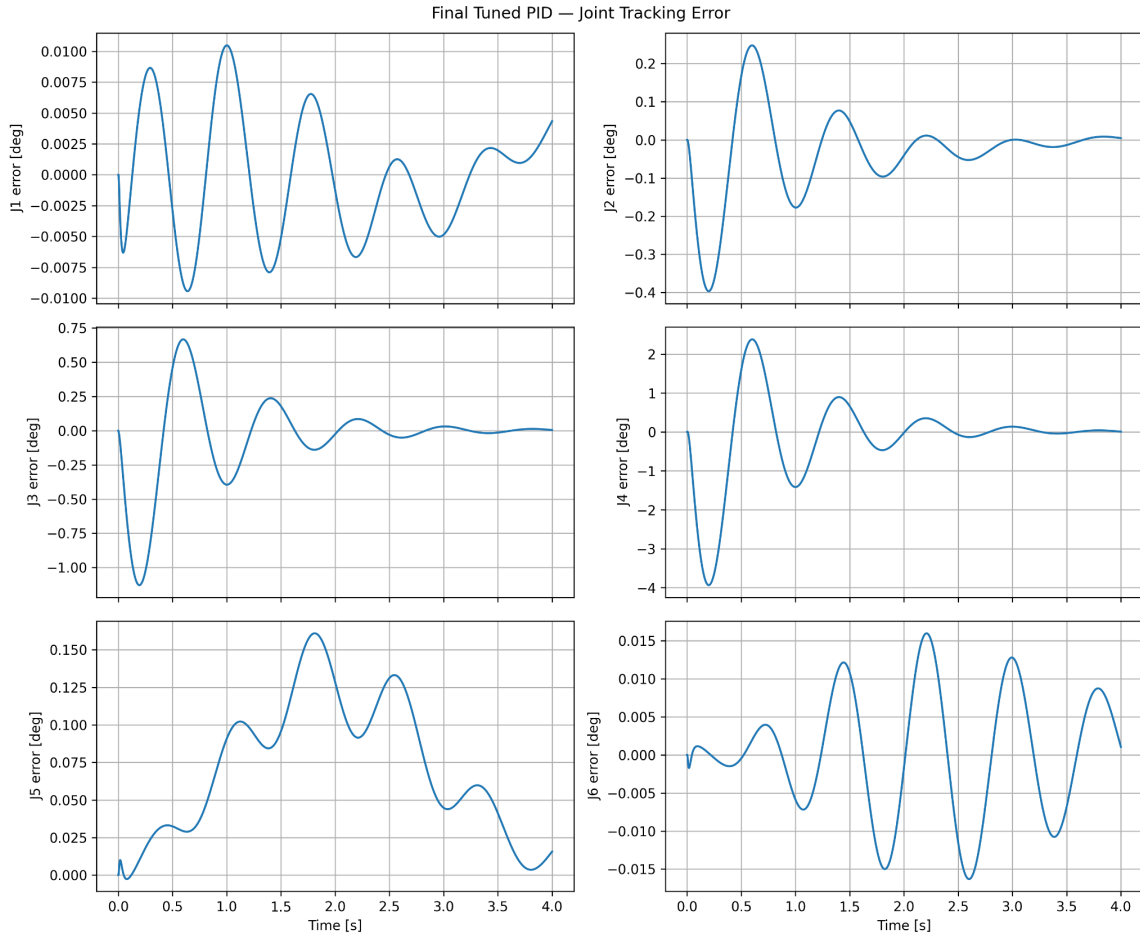


Figure 5: Final nominal PID joint errors. J4 is the dominant transient tracking-error channel.

The large J4 error initially looked like a possible saturation or coupling problem. I therefore decomposed the inverse-dynamics demand at the relevant transient instead of retuning blindly. Near the investigated peak, J4 demand was approximately -1.764 N m and was dominated by gravity (-1.763 N m); self-inertial and coupling contributions were only millinewton-meter scale. The PID command reached about -2.006 N m and J4 torque utilization remained only about 7.24%. The error was therefore not a torque-limit problem. It arose because the controller starts with zero tracking error and hence zero PID torque while the plant already requires nonzero gravity compensation. The feedback controller must first develop error before producing the balancing torque.

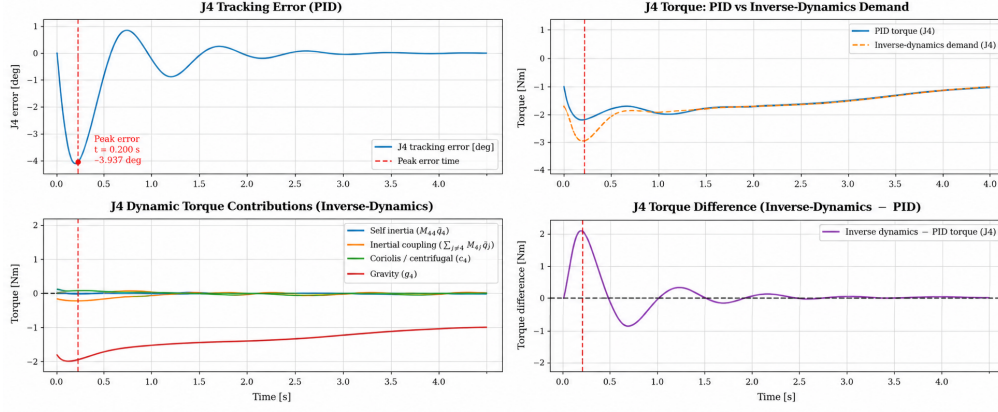


Figure 6: J4 dynamics decomposition used to identify gravity loading, rather than saturation, as the dominant origin of the PID transient.

This diagnosis is one of the reasons I retained PID as a baseline. Its limitation is physically interpretable and directly motivates the next controller rather than merely producing a worse number.

4. Computed-Torque Control

4.1 Formulation

Computed torque explicitly uses the nonlinear model:

$$v = \ddot{q}_{ref} + K_d(\dot{q}_{ref} - \dot{q}) + K_p(q_{ref} - q), \quad (3)$$

$$\tau = M(q)v + c(q, \dot{q}) + g(q). \quad (4)$$

The final nominal gains were $K_p = 400I$ and $K_d = 40I$, corresponding to $\omega_n = 20$ rad/s and $\zeta = 1$. Unlike the PID, the controller can generate the required gravity torque even when tracking error is nearly zero.

4.2 Nominal result

On the same nonlinear benchmark, computed torque achieved overall joint RMSE of only 0.001244° . TCP position RMSE/max/final were 0.004139/0.006550/0.000632 mm, and orientation RMSE/max/final were 0.002883/0.004539/0.000091°. The maximum torque vector was approximately

$$[0.2064, 40.7924, 20.2643, 1.8253, 0.1803, 1.28 \times 10^{-5}] \text{ Nm},$$

which closely reproduces the Part 1A inverse-dynamics demand. This is exactly what should happen when the controller model and simulated plant share the same rigid-body dynamics.

The result is therefore both impressive and deliberately treated with suspicion. Near-perfect nominal cancellation does not by itself demonstrate robustness; it demonstrates consistency between the plant and the model used by the controller. That distinction became the central question of R1.

The Python implementation required about 8.2 ms per controller update in the measured environment, largely because the current Coriolis calculation uses numerical differentiation. I do not interpret this as an intrinsic 8 ms cost of computed-torque control. It is an implementation cost of

the present reference model and would be inappropriate evidence for a general real-time feasibility claim.

5. Discrete LQR

5.1 Initial failure as a diagnostic result

The LQR study began with a continuous-time local linearization about the nominal start equilibrium. The first nonlinear trajectory test performed poorly. Instead of treating the failure as evidence that LQR was unsuitable, I traced the control law: a fixed equilibrium gravity torque was being used while the reference moved through configurations with changing gravity demand. The problem was not simply the feedback gain; the feedforward structure was inconsistent with the trajectory.

I then discretized the Part 1A linearization exactly under ZOH at 2 ms, solved the discrete algebraic Riccati equation, and normalized state/input penalties using Bryson-style allowable scales. A control-penalty sweep led to $\rho_u = 100$ for the final candidate. Most importantly, replacing fixed $g(q_{start})$ feedforward with $g(q_{ref})$ reduced nonlinear overall joint RMSE from approximately 0.810° to 0.0155° . I attribute that improvement primarily to correcting the feedforward architecture, not to LQR feedback alone.

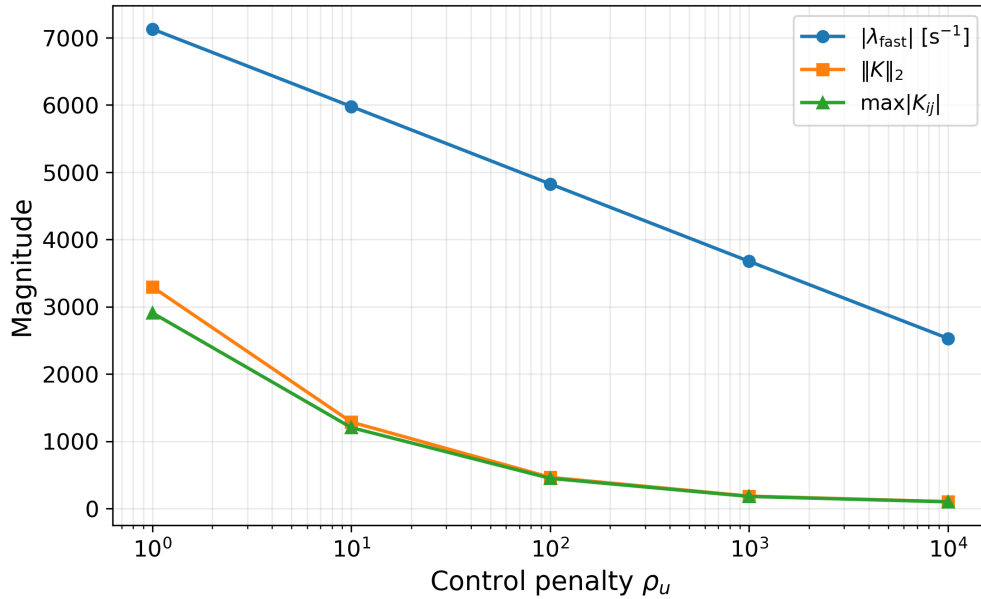


Figure 7: Discrete LQR control-penalty study. The sweep was used to connect penalty selection to closed-loop poles and feedback magnitude before nonlinear testing.

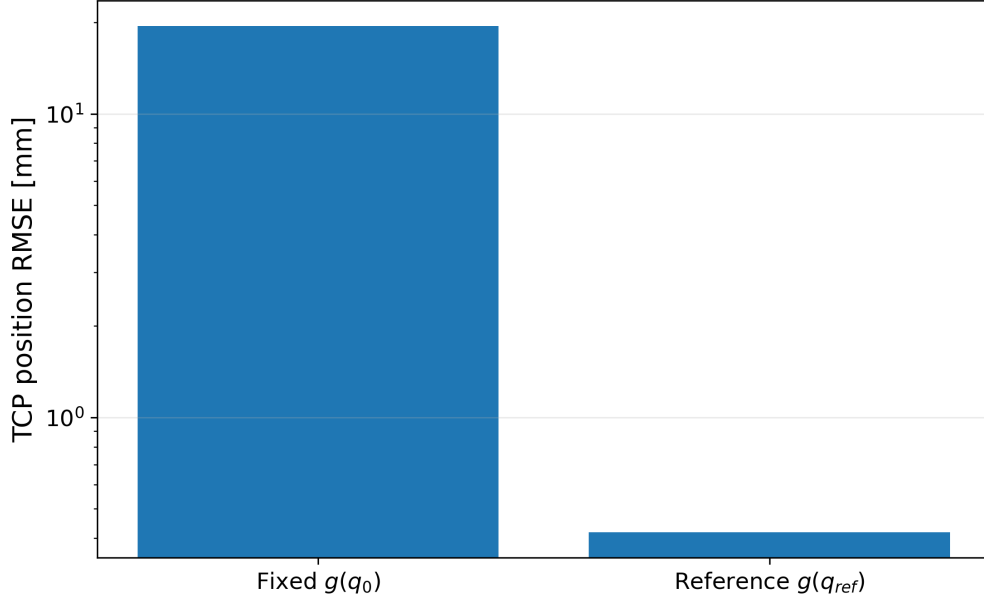


Figure 8: Effect of gravity-feedforward formulation on nonlinear TCP tracking. The result exposed a structural modeling issue that gain retuning alone would not have fixed cleanly.

5.2 Final nominal design

The final controller uses

$$\boldsymbol{\tau} = \mathbf{g}(\mathbf{q}_{ref}) - K \begin{bmatrix} \mathbf{q} - \mathbf{q}_{ref} \\ \dot{\mathbf{q}} - \dot{\mathbf{q}}_{ref} \end{bmatrix}, \quad (5)$$

with the fixed gain computed from the nominal start-equilibrium linearization and $\rho_u = 100$. The discrete closed-loop spectral radius is 0.9900498439.

Discrete LQR Joint-Space Trajectory Tracking

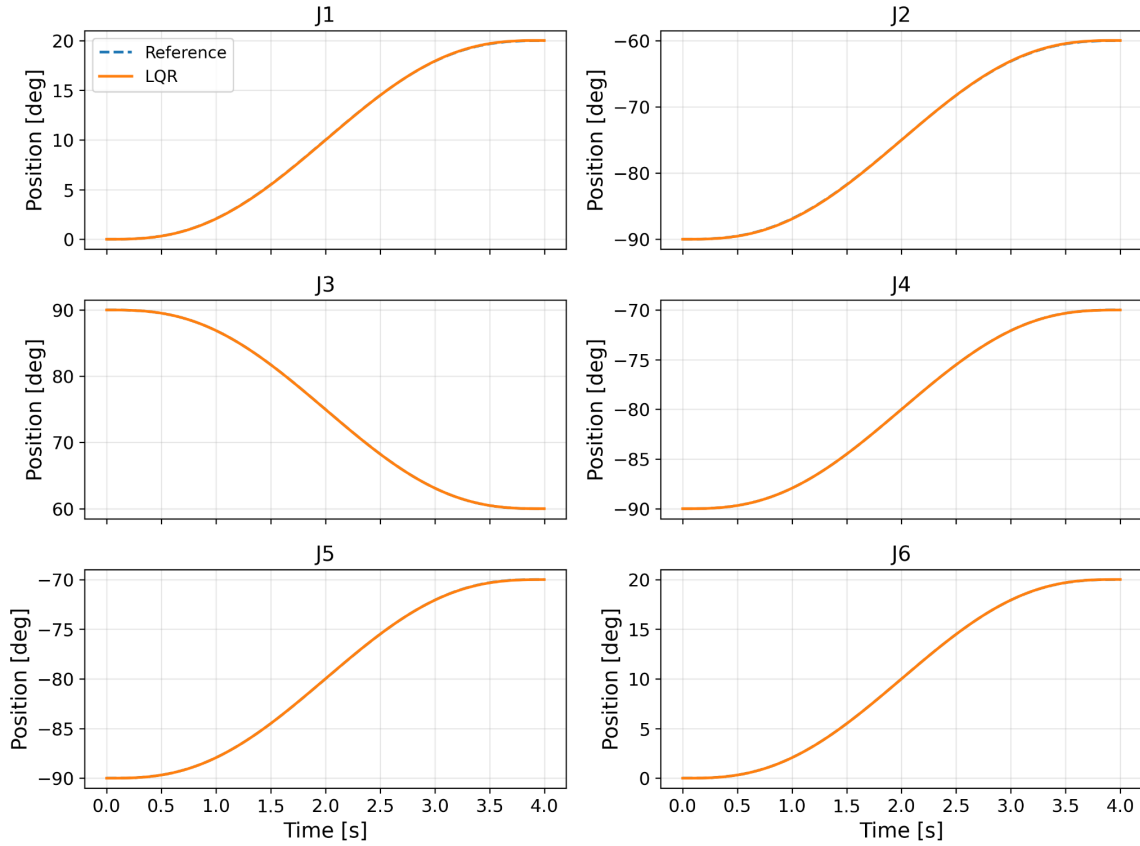


Figure 9: Final nominal discrete-LQR joint tracking on the nonlinear plant.

Overall joint RMSE was 0.015502° ; TCP position RMSE was 0.4185 mm and orientation RMSE was 0.03776° . Maximum torque remained near the nominal inverse-dynamics demand, with J2 at about 40.83 N m. The online Python feedback calculation averaged about 0.52 ms. A C++ implementation reproduced the online feedback/summation/saturation result to a maximum torque difference of approximately 4.55×10^{-13} N m when supplied with the same gravity feedforward.

The key boundary is that the gain still comes from one fixed local linearization. Reference-dependent gravity compensation corrects an important known nonlinear load, but it does not turn the controller into a globally linearized or gain-scheduled design.

6. Constrained Linear MPC

6.1 Prediction model and design choices

The MPC uses the same discrete local model as the LQR but optimizes a finite sequence of control corrections. In equilibrium coordinates $z = x - x_{eq}$ and $v = \tau - \tau_{eq}$,

$$z_{k+1} = A_d z_k + B_d v_k. \quad (6)$$

The trajectory-dependent gravity term is included as a known affine feedforward shift, while the optimized correction is constrained by the adopted joint torque limits. The final design uses prediction horizon $N_p = 160$ (320 ms), control horizon $N_c = 40$ (80 ms), $\rho_u = 10$, and OSQP absolute/relative tolerances of 10^{-6} .

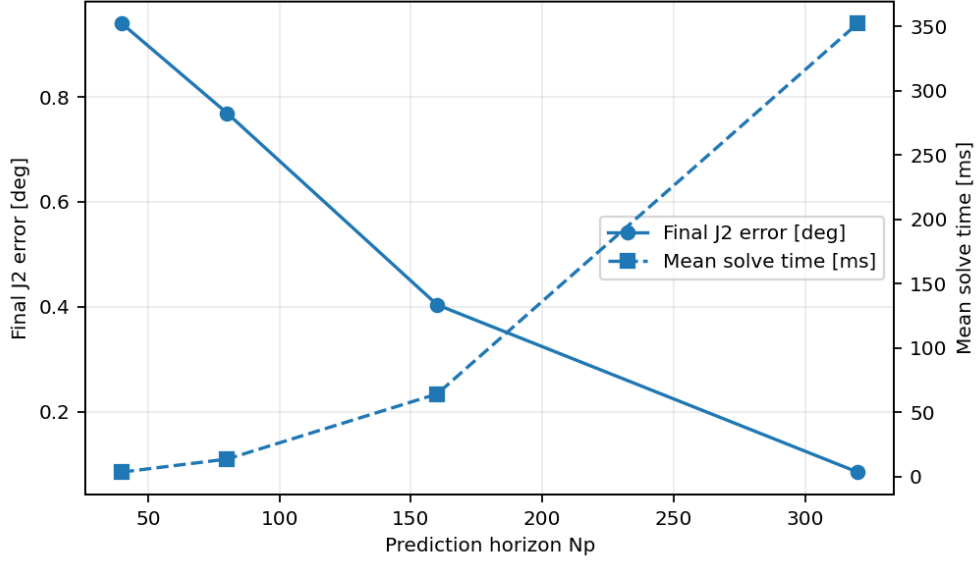


Figure 10: Prediction-horizon study. Horizon selection was based on both tracking behavior and the rapidly increasing optimization cost.

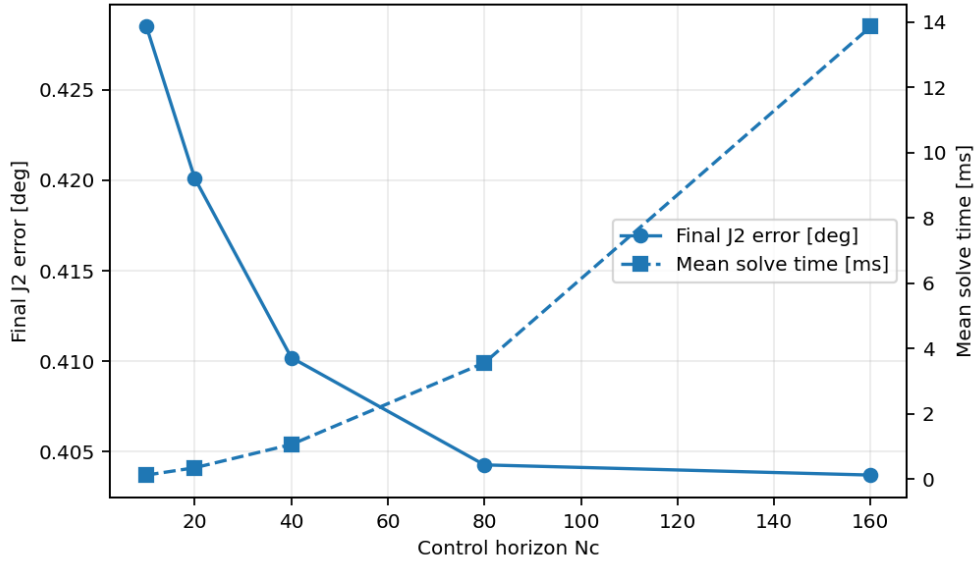


Figure 11: Control-horizon reduction. Holding the last optimized correction reduced decision dimension while preserving the tested tracking behavior closely.

6.2 Runtime architecture as part of controller design

The first nominal Python benchmark required about 17.97 ms per update. Profiling showed that the online loop repeatedly evaluated matrix products whose factors were fixed by the model or known reference. I therefore rewrote the QP gradient

$$q_k = 2G_c^T \bar{Q} (Fz_k + GV_{ff,k} - Z_{ref,k})$$

as

$$q_k = K_x z_k + b_k, \quad K_x = (2G_c^T \bar{Q})F, \quad b_k = (2G_c^T \bar{Q})(GV_{ff,k} - Z_{ref,k}),$$

and precomputed K_x and the known-reference sequence b_k . This did not change the QP; it changed where the algebra was executed.

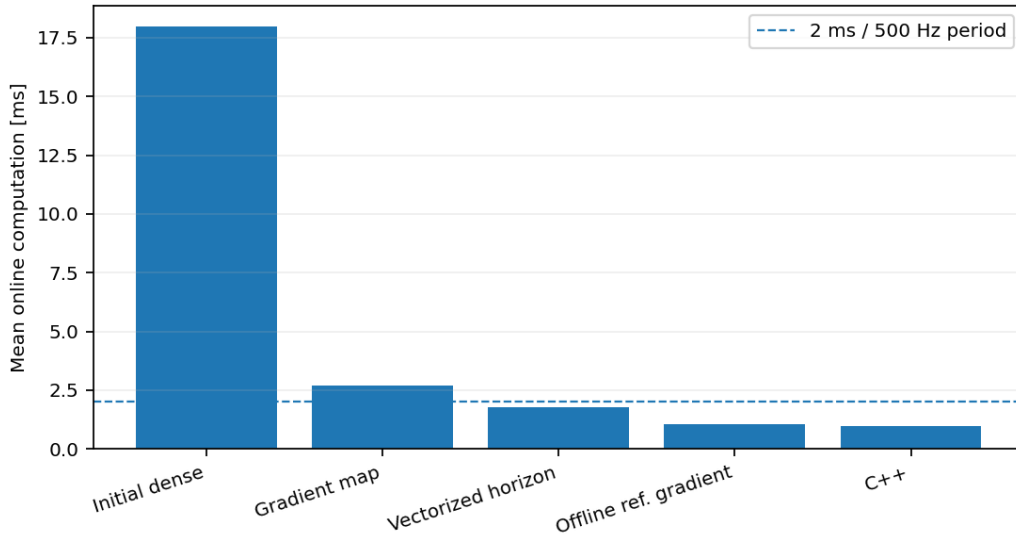


Figure 12: MPC implementation progression. Profiling and offline/online algebraic separation produced most of the runtime reduction before the final C++ step.

The final nominal MPC achieved overall joint RMSE 0.048474° , TCP position RMSE 0.3784 mm, and orientation RMSE 0.1298° . No nominal sample activated a torque constraint. Therefore the controller *contains* explicit constraint handling, but this nominal trajectory does not demonstrate behavior under active constraints.

The optimized Python implementation averaged roughly 1.03–1.08 ms per update. C++/Eigen/OSQP reproduced the Python torque sequence with maximum absolute difference 2.03×10^{-10} N m and mean runtime 0.9605 ms. The approximately 9.95 ms maximum C++ outlier prevents a hard-real-time claim.

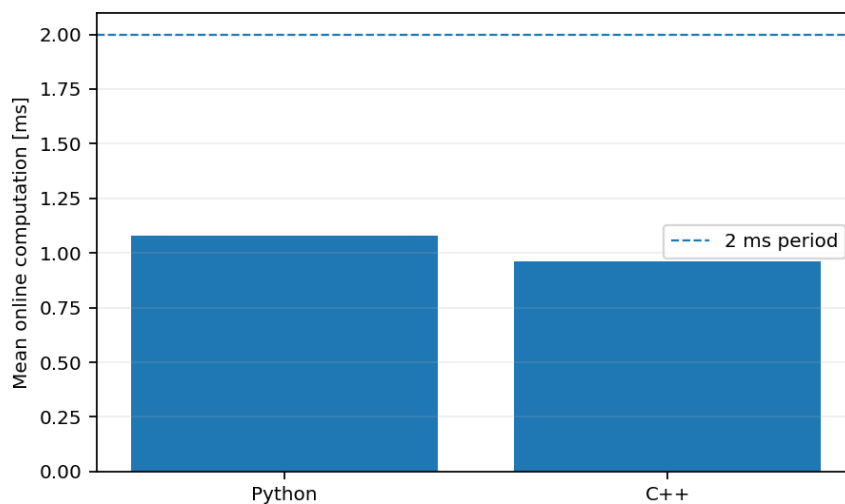


Figure 13: Measured mean runtime of the final optimized Python and C++ MPC implementations relative to the 2 ms benchmark period.

7. Nominal Cross-Controller Assessment

Table 2: Nominal nonlinear benchmark summary.

Controller	Joint RMSE [deg]	TCP pos. RMSE [mm]	TCP ori. RMSE [deg]	Peak util. [%]
PID	0.469844	5.835795	1.527910	27.169
Computed torque	0.001244	0.004139	0.002883	27.195
LQR	0.015502	0.418511	0.037763	27.218
MPC	0.048474	0.378424	0.129772	27.174

The nominal table should not be read as a universal ranking. Computed torque benefits from exact agreement between controller and plant model. LQR and MPC use one fixed local prediction/feedback model plus trajectory-dependent gravity feedforward. PID deliberately uses no plant model. Their nominal errors therefore measure different combinations of controller capability and assumption validity. The nearly identical peak torque utilization across controllers is useful: differences in tracking are not being driven by one controller simply using much more of the adopted actuator envelope.

8. Robustness Methodology

I used two perturbations because “robustness” is not one failure mode. A payload changes the plant continuously and invalidates the nominal inertia/gravity model. A short external torque pulse leaves the model unchanged before and after the event but injects an unmeasured generalized force during the event. I kept every controller tuning parameter fixed so the tests measure performance retention rather than retuning skill.

8.1 R1: persistent 2 kg payload/model mismatch

The actual plant was augmented by a 2.0 kg point mass at tool-frame $[0, 0, 0.10]$ m. Its translational Jacobian contribution was used to augment the mass matrix and gravity vector, with Coriolis terms recomputed from the perturbed mass matrix. Payload rotational inertia was intentionally omitted, so this is a controlled point-mass mismatch rather than a full payload identification model. At the nominal start pose, the payload increased J2 gravity compensation by approximately 9.65 N m and J4 by about 1.96 N m. Controllers continued to use the bare-arm model/gains.

8.2 R2: transient external Joint 2 disturbance

The nominal bare-arm plant received

$$\tau_{dist}(t) = [0, 20, 0, 0, 0, 0]^T \text{ N m}, \quad 2.0 \leq t < 2.2 \text{ s.}$$

The disturbance was added only to the actual plant input and was not supplied to the controller. I chose J2 because it carries the dominant nominal load; 20 N m is large enough to create a visible tracking deviation but does not intentionally turn the test into saturation.

9. Robustness Results

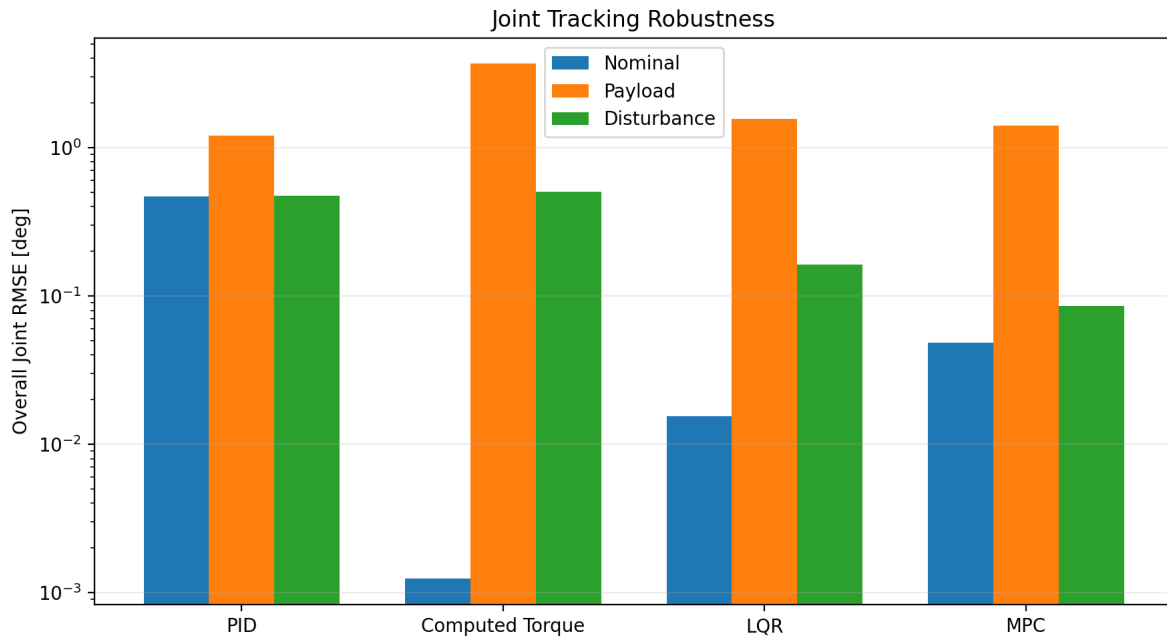


Figure 14: Overall joint RMSE under nominal, persistent payload mismatch, and transient external disturbance. Log scaling is necessary because nominal computed-torque error is several orders of magnitude below its perturbed results.

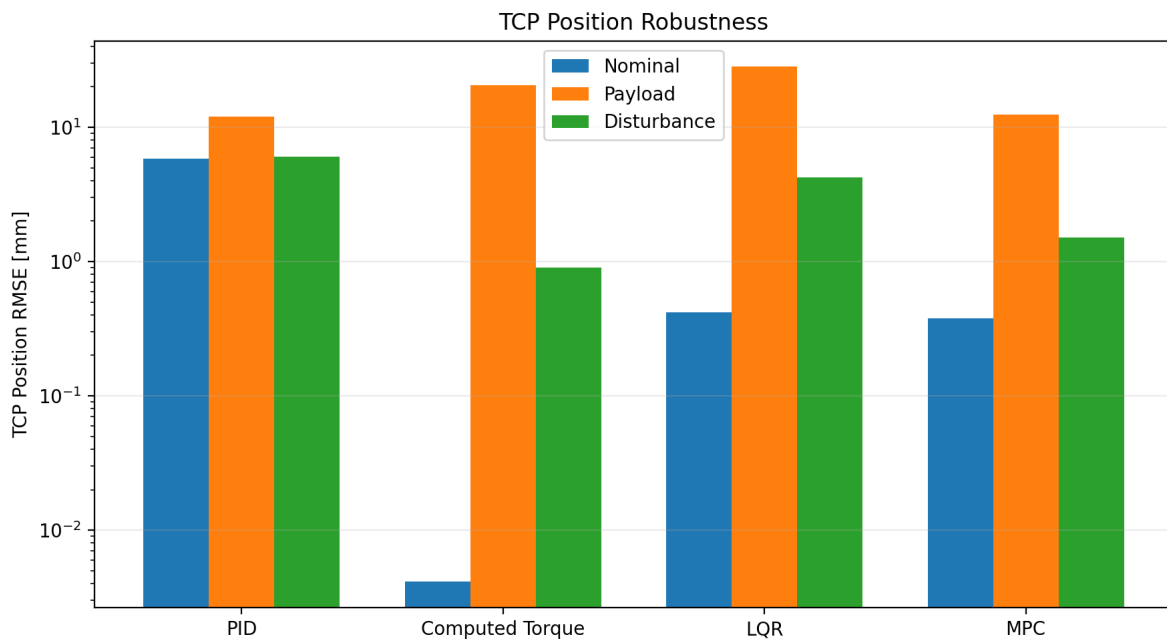


Figure 15: TCP position RMSE under the three conditions. Cartesian evaluation reveals effects that are not always obvious from aggregate joint RMSE alone.

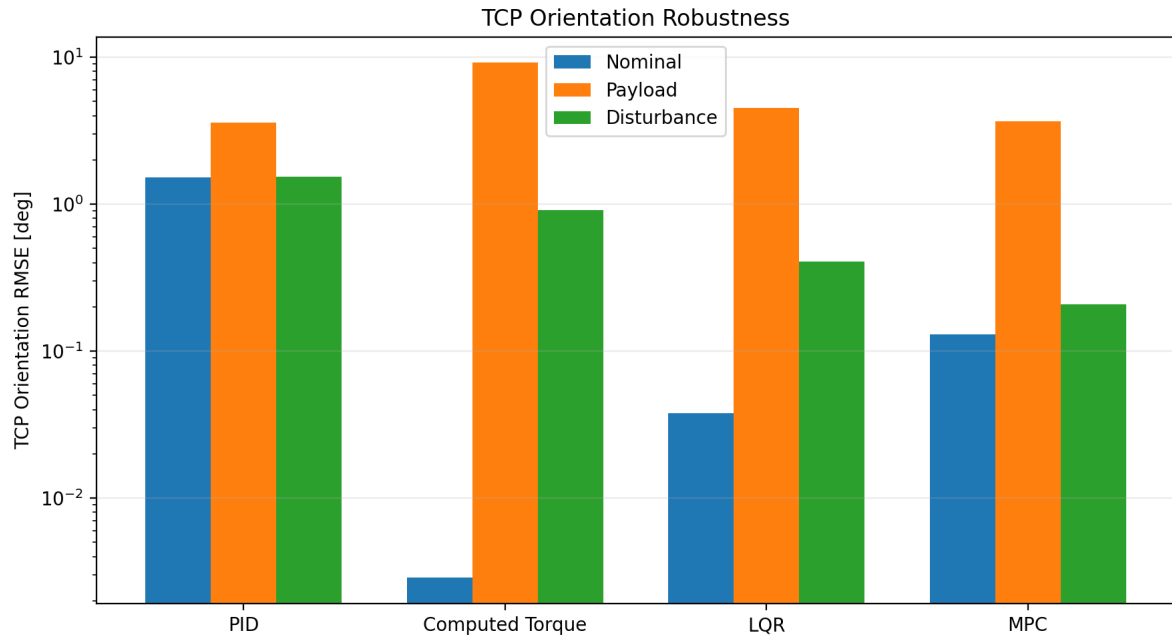


Figure 16: TCP orientation RMSE under the three conditions.

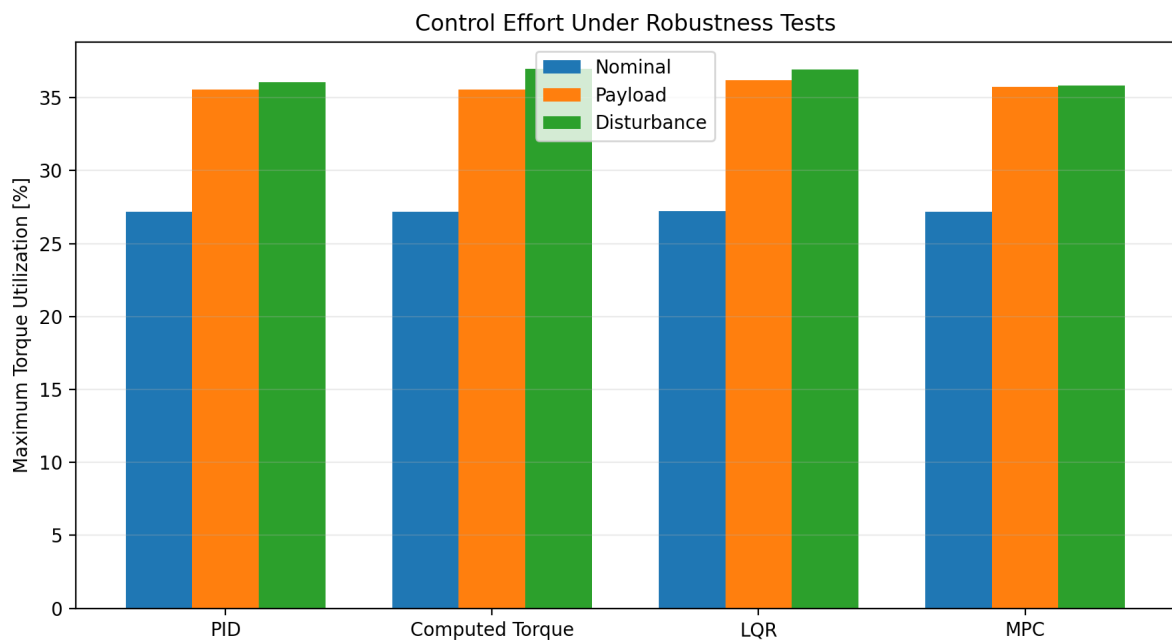


Figure 17: Peak commanded-torque utilization. Neither robustness case drove the controllers near the adopted torque constraints; performance degradation therefore cannot be explained primarily by saturation.

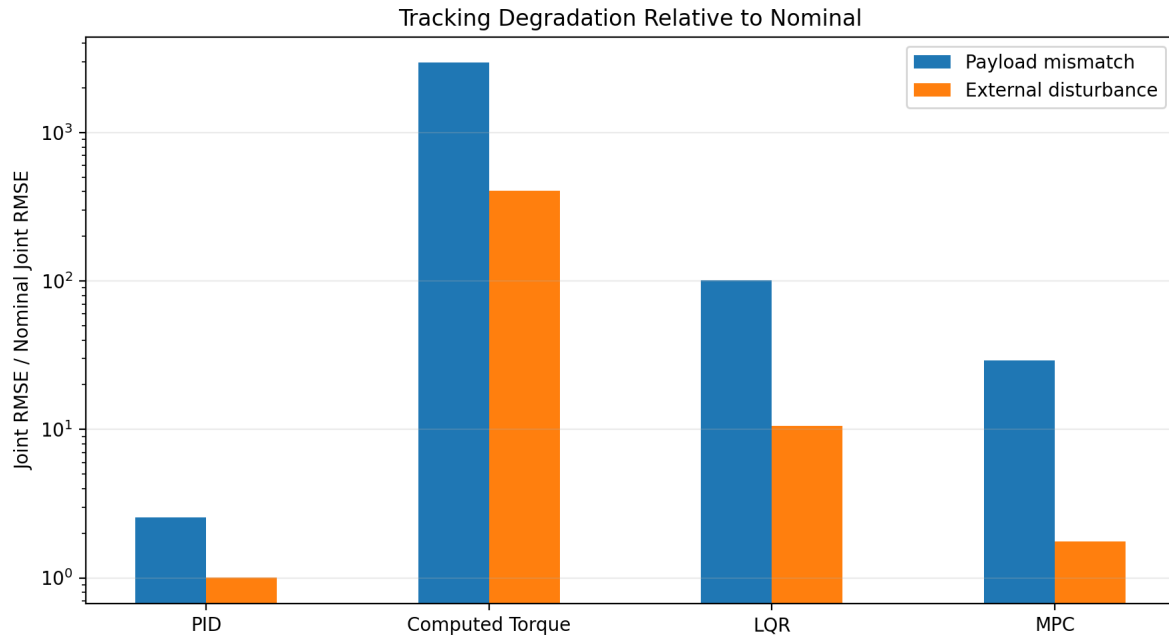


Figure 18: Joint-RMSE degradation relative to each controller’s own nominal result. The logarithmic ratio is useful for performance retention, but must be read together with absolute error because tiny nominal denominators produce very large ratios.

Table 3: Robustness summary from the common comparison script.

Controller	Case	Joint RMSE [deg]	TCP pos. RMSE [mm]	TCP ori. RMSE [deg]	Peak util. [%]
PID	Nominal	0.469844	5.835795	1.527910	27.169
PID	Payload	1.205309	11.963438	3.585649	35.547
PID	Disturbance	0.472416	6.032206	1.531076	36.044
Computed torque	Nominal	0.001244	0.004139	0.002883	27.195
Computed torque	Payload	3.684475	20.562134	9.174375	35.541
Computed torque	Disturbance	0.504513	0.900220	0.907051	36.979
LQR	Nominal	0.015502	0.418511	0.037763	27.218
LQR	Payload	1.564289	28.293832	4.533704	36.196
LQR	Disturbance	0.163066	4.246570	0.406142	36.950
MPC	Nominal	0.048474	0.378424	0.129772	27.174
MPC	Payload	1.412378	12.380191	3.659589	35.762
MPC	Disturbance	0.085670	1.503611	0.209132	35.824

9.1 What R1 reveals

The payload test is the clearest demonstration that nominal accuracy and robustness are different properties. Computed torque changed from 0.001244° to 3.684° overall joint RMSE and from 0.00414 to 20.56 mm TCP position RMSE. This is consistent with the structure: the same model that provides near-perfect nominal cancellation becomes the mechanism through which persistent model error enters the commanded torque. The very large degradation ratios (thousands of times nominal) are real mathematically but should not be used alone because the nominal denominator is exceptionally small.

PID degraded much less in relative terms ($2.57\times$ joint RMSE) because it did not depend on an explicit bare-arm inverse model. That does not make its absolute tracking best: payload joint RMSE remained 1.205° and TCP position RMSE 11.96 mm. The useful engineering conclusion is narrower: the PID baseline gives up nominal precision but is less exposed to this specific model-cancellation error.

LQR and MPC both retained their nominal controller models and gravity/feedforward assumptions, so both degraded under the persistent payload. LQR produced 1.564° joint RMSE and 28.29 mm TCP position RMSE; MPC produced 1.412° and 12.38 mm. These results show that the fixed nominal model remains a significant dependency even when the controller is formulated as optimal feedback or finite-horizon optimization.

9.2 What R2 reveals

The disturbance test tells a different story because the perturbation disappears after 0.2 s. Computed torque experienced a substantial transient (0.5045° aggregate joint RMSE), but its final TCP position error returned to approximately 0.000632 mm, essentially its nominal final value. The controller model becomes correct again when the external force disappears, so the model-based cancellation structure can recover its nominal behavior.

PID's full-trajectory RMSE changed very little, from 0.4698° to 0.4724° . This does not mean the disturbance had no effect; the nominal PID already carries a comparatively large transient-error baseline, so a short disturbance contributes little to a four-second aggregate RMSE. This is why I retained joint-specific and TCP metrics rather than relying on one scalar score.

Under this specific J2 pulse, LQR joint RMSE increased to 0.1631° and TCP position RMSE to 4.247 mm. MPC increased to 0.08567° and 1.504 mm. I report this as a condition-specific result, not a general statement that MPC is more robust than LQR. Different disturbance channels, horizons, weights, or model errors can change that relationship.

10. Computation and Implementation Assessment

Table 4: Representative nominal controller computation measurements.

Controller	Mean [ms]	Relative to 2 ms	Interpretation
PID	0.026	1.3%	Simple feedback baseline
Computed torque	8.25	412%	Current Python numerical-dynamics implementation; not intrinsic algorithm cost
LQR	0.52	26%	Fixed-gain feedback plus gravity feedforward
MPC, optimized Python	1.03	51%	Mean below period; outliers observed
MPC, C++/OSQP	0.9605	48%	Mean/P99 favorable; max outlier prevents hard-RT claim

I deliberately separate mathematical controller complexity from implementation cost. The computed-torque Python result is dominated by the current numerical Coriolis implementation, while the MPC study demonstrated that poor online algebra placement can dominate solver cost. Conversely, a low mean runtime is not sufficient for hard-real-time certification. These distinctions matter more than a single timing number.

11. Cross-Controller Engineering Assessment

The four controllers expose four different engineering trade-offs. PID is transparent, inexpensive, and comparatively insensitive to explicit model mismatch, but gravity loading appears directly as tracking error unless integral action or feedforward compensates it. Computed torque can nearly remove nominal nonlinear dynamics when the model is correct, but that strength is also its vulnerability under persistent model error. LQR gives efficient optimal state feedback from a local model, but the study showed that a correct feedback gain cannot compensate for a structurally wrong feedforward assumption. MPC adds prediction and explicit constraints, but its value depends on implementation discipline and on whether the operating condition actually exercises those constraints.

The most important outcome is methodological. I did not accept a controller because its nominal plot looked good, nor reject one because its first implementation failed. I used the model to form a hypothesis, designed a test that isolates the suspected mechanism, inspected the result, and changed the design only after the cause was identifiable. The J4 PID decomposition, LQR gravity-feedforward diagnosis, MPC profiling/restructuring, and R1/R2 separation are all examples of the same process.

12. Limitations and Reproducibility Boundaries

- All analytical controller tests use the same ideal fixed-base rigid-body plant family; Part 2 Isaac Sim is required as an independent simulation layer.
- The 2 kg payload is modeled as a point mass at tool-frame +Z 0.10 m; payload rotational inertia is omitted intentionally.
- R1 and R2 are representative perturbations, not an exhaustive robustness proof. No formal robust-stability margin is claimed.
- The disturbance is one +20 N m J2 rectangular pulse. Results should not be generalized to arbitrary disturbance spectra or channels.
- Torque limits are adopted simulation constraints from the imported model, not independently verified physical motor ratings.
- No nominal or robustness case activated the MPC torque constraints. Constraint handling is implemented but not yet stress-demonstrated.
- LQR and MPC use one fixed linearization about the nominal start equilibrium; neither is nonlinear or gain-scheduled.
- Reference-dependent gravity/feedforward terms assume the planned reference is known. Online reference changes require recomputation of reference-dependent terms.
- Python-C++ agreement demonstrates implementation consistency, not physical truth. Physical/model independence comes from the earlier Isaac cross-validation and the upcoming Part 2 closed-loop simulation.
- Runtime measurements were made on a general-purpose Linux system and do not establish hard-real-time deadline guarantees.

13. Reproducibility Map

Purpose	Script / artifact
PID controller	<code>controllers/pid.py</code>
PID nominal benchmark	<code>analysis/pid_final_benchmark.py</code>
Computed-torque controller	<code>controllers/computed_torque.py</code>
Computed-torque nominal benchmark	<code>analysis/computed_torque_benchmark.py</code>
LQR controller / benchmark	<code>controllers/lqr.py</code> ; <code>analysis/lqr_nominal_benchmark.py</code>
MPC controller / benchmark	<code>controllers/mpc.py</code> ; <code>analysis/mpc_nominal_benchmark.py</code>
Payload plant perturbation	<code>dynamics/payload_dynamics.py</code>
External disturbance model	<code>dynamics/disturbance_dynamics.py</code>
R1 controller benchmarks	<code>analysis/*_payload_robustness.py</code>
R2 controller benchmarks	<code>analysis/*_disturbance_robustness.py</code>
Common robustness comparison	<code>analysis/robustness_comparison.py</code>
Robustness data	<code>results/robustness/</code>

14. Conclusion and Transition to Part 2

Part 1B converted the characterized UR5e plant into a controlled sequence of controller-design questions. PID established the no-model baseline and exposed gravity-driven tracking error. Computed torque demonstrated the precision available from accurate nonlinear cancellation and, through the payload test, the cost of depending on that accuracy. LQR showed that a local optimal gain can work well on the nonlinear benchmark only when the feedforward structure is consistent with the moving reference. MPC added finite-horizon prediction and explicit constraints, while its implementation study showed that runtime architecture can matter as much as solver choice.

The robustness tests then broke the nominal assumptions in two different ways. Persistent payload mismatch damaged all model-dependent designs and most dramatically exposed computed torque's cancellation sensitivity. A transient J2 disturbance produced a different pattern: the model-based controllers could recover once the unmodeled force disappeared, while aggregate metrics had to be interpreted relative to each controller's nominal error floor. Across both tests, torque utilization remained well below the adopted limits, so the observed degradation reflects controller/model interaction rather than simple saturation.

I therefore consider the analytical controller study complete enough to proceed. The next stage is not more tuning against the same equations. Part 2 should move the selected implementations into Isaac Sim and ask whether the conclusions survive an independent simulation environment, including its articulation dynamics, numerical integration, joint-drive interface, and timing behavior. That transition preserves the project principle established from the beginning: *validate the model before trusting the controller, then validate the controller outside the model used to design it.*